ACADEMIA ENGINEERING

Research Article
Published: 10 June 2026
https://doi.org/10.20935/AcadEng8332

A new DCache prefetch algorithm based on branch prediction information for AI industrial engineering

Ding Ning[1,*], Ray Wai Man Kong[1,2], Shien Ping Feng[1]

Academic Editor: Chuan Heng Foh

Abstract

Introduction: This work proposes a novel cache prefetching algorithm and corresponding microarchitecture for high-performance Central Processing Unit (CPU) designs. Existing stride-based and conventional DCache prefetching schemes have limited prediction accuracy for loop-intensive workloads, fail to fully exploit branch prediction metadata, and easily induce cache pollution and unnecessary memory bandwidth and power consumption—these constitute the key research gaps addressed in this work. This study aims to leverage branch prediction information to optimize DCache prefetching, accurately identify program loop structures, and reduce cache miss rates while avoiding excessive redundant prefetch operations.

Materials and methods: The research methodology adopts the Gem5 simulation platform to conduct comparative experiments across typical workloads; we design a dedicated branch history table (BHT) replacement strategy with repetition and usage counters, define clear update rules and loop detection thresholds, and adopt statistical analysis, including confidence intervals, variance and significance testing, to evaluate performance.

Results: Quantitative experimental results show that the proposed scheme effectively reduces the DCache miss rate by 2.443%, achieves stable Instructions Per Cycle (IPC) performance by 0.4716 instructions per cycle with negligible overhead, and significantly reduces unnecessary data prefetching, cache pollution and power cost compared with traditional stride prefetching.

Conclusions: The optimized design is well-suited for general-purpose CPU scenarios as well as AI training and inference workloads dominated by loop-parallel computation, offering a lightweight and high-performance alternative to existing data prefetching approaches.

Keywords: *cache, branch prediction, data prefetching, central processing unit (CPU), CPU performance*

Citation: Ning D, Kong RWM, Feng SP. A new DCache prefetch algorithm based on branch prediction information for AI industrial engineering. *Academia Engineering* 2026;3. https://doi.org/10.20935/AcadEng8332

1. Introduction

1.1. DDR and Cache

One of the Central Processing Unit (CPU) bottlenecks in performance is sending the correct data and instructions to the CPU in modern CPU architectures. As shown in **Figure 1**, a traditional CPU fetches instructions and data from Double Data Rate Synchronous Dynamic Random-Access Memory (DDR SDRAM). Double Data Rate Synchronous Dynamic Random-Access Memory (DDR SDRAM, also called DDR) is a type of Synchronous Dynamic Random-Access Memory (SDRAM) widely used in computer systems and other electronic devices. The access speed of DDR affects computer performance because it provides the data and instructions that computers need, and computers cannot function without them. However, DDR contains a vast amount of data, and access speed will be very slow because larger memory cells take longer to access. Modern CPUs can process several pieces of data in a single cycle, but the DDR may take hundreds of cycles to provide even a single piece of data.

To solve this problem, several cache levels are introduced between the DDR and CPU, as shown in **Figure 2**. The L1–L3 (Level 1–Level 3) Cache, shown as a dotted line, is optional, and some systems may have only one or two caches. The size of DDR is the biggest (8–128 GB for DDR5 standard in most computer systems), but the access speed is quite slow since it takes more time to access the bigger memory. On the other hand, the L1 cache is the smallest, while the access speed is the fastest, for the same reason as DDR. The size and speed of the L2 and L3 caches are between those of DDR and L1. Overall, the rule for memory size is DDR > L3 > L2 > L1, while the rule for memory access speed is DDR < L3 < L2 < L1. For these reasons, the low-level cache prefetches some data from the high-level cache or DDR. Ishii introduced a method of instruction in his paper [1]. The prefetching algorithm is significant for CPU performance because once a cache miss occurs (the CPU cannot find the data in the L1 cache), waiting for the data to be fetched can take a long time.

[1]Department of Systems Engineering, City University of Hong Kong, Hong Kong, China.
[2]Eagle Nice (International) Holdings Ltd., Hong Kong, China.
*email: dning7-c@my.cityu.edu.hk

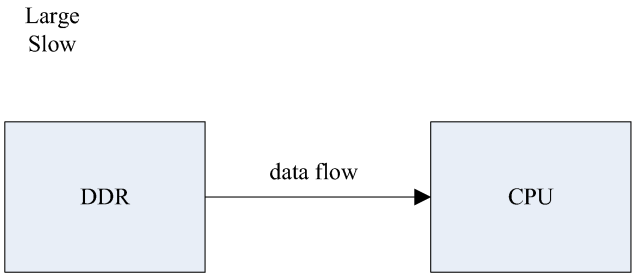


Figure 1 • Double Data Rate Synchronous Dynamic Random-Access Memory (DDR) and Central Processing Unit (CPU).

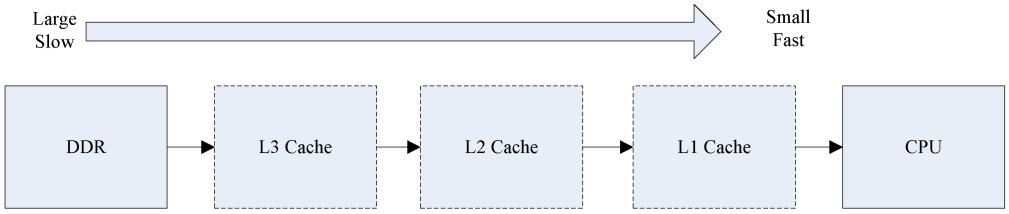


Figure 2 • Cache between Double Data Rate Synchronous Dynamic Random-Access Memory (DDR) and Central Processing Unit (CPU). L3: Level 3 Cache; L2: Level 2 Cache; L1: Level 1 Cache; black arrow: Data flow; grey arrow: Size decreased/Speed increased.

To enhance caching efficiency, cache prefetching and data and instruction prediction are widely used. The data and instruction prediction functions are based on two key factors:

(1) Time dependency: The data and instructions used before will likely be used again in a very short time. For this reason, the most recent data can be stored to avoid cache misses.

(2) Spatial dependency: When the CPU uses one piece of data, it will likely use nearby data. For this reason, once the CPU accesses data, nearby addresses can be prefetched into the cache to avoid cache misses in future operations.

There are two kinds of cache in a CPU: Data Cache (DCache), which stores data, and Instruction Cache (ICache), which stores instructions. The Load and Store Unit (LSU) is an important module in the CPU that fetches data into and out of the CPU. It also provides DCache with time and space information for prefetching data. At the same time, the Instruction Fetch Unit (IFU), another important module in the CPU, focuses on branch prediction. When the IFU provides branch prediction information, ICache uses it to prefetch instructions. The prefetching operation of DCache and ICache can significantly reduce the possibility of cache misses in most CPU systems. **Figure 3** shows the positions of DCache and ICache in the CPU architecture.

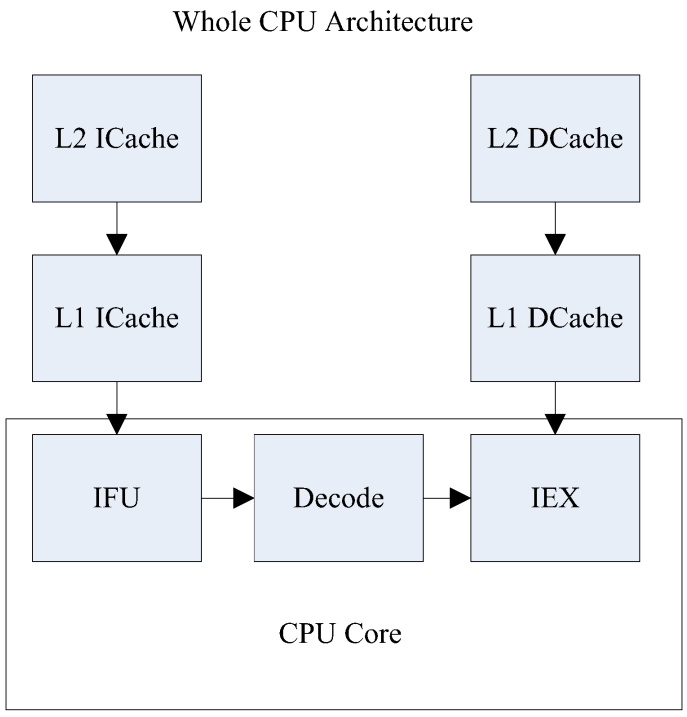


Figure 3 • Cache in Central Processing Unit (CPU) architecture. IFU: Instruction Fetch Unit; IEX: Instruction Execute; L1: Level 1; L2: Level 2; arrows: Data flow.

Currently, most CPU systems use only temporal and spatial dependency information for prefetching data into the DCache, while other valuable information is often overlooked. For instance, branch prediction information is widely used in the ICache but not in the DCache. In the following sections, we explore how to leverage branch-prediction information for DCache prefetching to address this current limitation.

1.2. Branch prediction

Firstly, branch prediction is a widely used technology in modern CPU architecture. Branch prediction technology uses historical information to predict the instruction address and prefetch the instruction before it is used. Most CPUs contain an ICache that can store instructions before they are needed, saving the time it takes to fetch them. To enhance system efficiency, a technology called prefetching can "guess" the next program counter (PC) and prefetch instructions into the ICache ahead of time. A module named IFU predicts the instruction address and provides ICache with the prediction information for prefetching instructions using branch prediction technology.

The Branch Target Buffer (BTB) submodule, a crucial component of the IFU, stores vital historical branch information. This includes the branch's direction (whether taken or not, introduced by Juan [2] and Yeh [3] in their paper) and the target address if the branch is taken. These two pieces of information are instrumental in instruction and data prefetching, underscoring the BTB's pivotal role in CPU architecture.

ICache is used for prefetching and storing instructions to enhance CPU performance; similarly, DCache is used for prefetching and storing data. However, most advanced CPUs currently use only IFU information to predict the prefetch address for ICache, not for DCache. DCache can only prefetch data based on the time and spatial dependency information provided by LSU.

However, the running instruction also has some relationship with the loading data. For example, in these two particular cases, the trace of the CPU PC is predicted quite accurately:

(1) When there is no branch instruction in the following sequence of instructions, the PC will always run in order in the address range of these instructions.

(2) When the CPU runs to an unconditional direct branch, it must jump to a fixed address. For this reason, the next PC can be predicted accurately.

While most methods of prefetching data into a cache rely on data time and spatial dependencies, the untapped potential of using branch prediction information is an exciting prospect. This could renew the DCache prefetching methodology, opening up new avenues for improving CPU performance.

When a branch's direction and jump target are confirmed, the next instruction is also confirmed. On the other hand, the instructions taken highly affect the load data from LSU. Thus, the branch prediction result can be used as the DCache prefetching information to achieve high performance.

Branch prediction technology is widely used in current CPU architectures [4, 5]. As mentioned in the background, branch prediction is implemented in the IFU and can provide two pieces of information: the taken direction and the taken address. These two pieces of information show the potential PC address, which can help with instruction prefetch. However, this information can also be used for data prefetching, since the CPU accesses data that depends on the instruction.

To analyze the dependency between the data and the instruction, the instruction type is identified in the next chapter.

1.3. Branch instruction and non-branch instruction

The CPU performs its operations by executing instructions one at a time. In some cases, when the CPU finishes an instruction, the PC is incremented by four to fetch the next instruction. This is an in-order case, which means the PC will proceed to the next stage by adding four. However, in another case, the PC of the CPU will not proceed in this way. The most special instruction of the CPU is a branch instruction, which may cause the PC to jump to another address instead of the current one (PC + 4). When the CPU encounters a branch instruction, it updates the PC to point to the next stage to fetch the correct instructions. **Figure 4** shows the meaning of branch instructions.

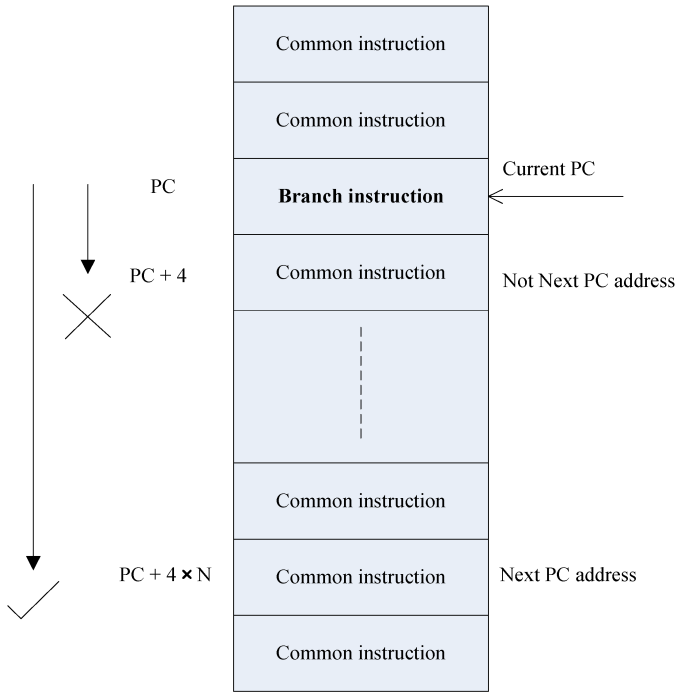


Figure 4 • Branch instruction. PC: Program Counter; arrows: PC running path; X symbol: The PC will not jump to this address; checkmark symbol: The PC will jump to this address; dashed line: represent serval instruction space.

The IFU focuses on branch instruction prediction and its next destination, as this instruction can define the PC address route. The branch instruction contains many attributes:

Unconditional direct branch instruction: The branch instruction always jumps to a target address without any conditions, and the target is written in the instruction.

Conditional direct branch instruction: The branch instruction jumps to a target address if it meets some conditions, and the target is written in the instruction. The jump condition may be a control or general register value.

Unconditional indirect branch instruction: The branch instruction always jumps to a target address without any conditions, and the sum of a general register and an offset gives the target. The offset may be 0 or written in the instruction.

Conditional indirect branch instruction: The branch instruction jumps to a target address if the CPU status meets some conditions and the sum of a general register and an offset gives the target. The offset may be 0 or written in the instruction. The jump condition may be a control register value or a general register value. It is worth noting that most CPUs do not support this branch type, making your understanding of it unique.

Table 1 shows the four types of branch instruction.

Table 1 • Branch instruction type.

Branch type	Unconditional	Conditional
Direct	1	3
Indirect	2	4

An unconditional branch instruction means the branch will always jump. In contrast, a conditional branch instruction means we should consider conditions (for example, a register value) to decide whether to jump. Direct branch instructions contain the jump address (target) of the next stage, whereas indirect branch instructions do not have this information. The register value or other status determines the target of the next stage for an indirect branch instruction. Since type 4 is seldom implemented in advanced CPU architectures, this article will focus on types 1–3.

Unconditional direct branch instruction: This branch instruction always jumps and clearly defines the target. The information is relatively straightforward, so the hardware system does not need to guess. Branch information is strongly dependent on instruction prefetching, so we can determine the next instruction the CPU fetches without guessing. Conversely, the data the CPU uses in the next step depends heavily on the next instruction. A memory or buffer should be built to store the dependency between the instruction and prefetched data, thereby enhancing the prefetch data hit rate.

Conditional direct branch instruction: When the branch is taken, the taken target is clearly defined in the instruction. However, the uncertainty lies in whether to take the branch. This is where the TAgged GEometric history length (TAGE) memory [6–8] plays a crucial role. Storing the branch history information enables us to predict the direction taken. With the aid of TAGE information, the direction prediction rate can reach nearly 98% [7] once the CPU system has warmed up. This term refers to the location where most programs have been run and to the branch history information stored in the TAGE memory. At this stage, the CPU system can establish a strong dependency between the branch instruction and the used data.

Conditional indirect branch instruction: TAGE memory can be used to predict the directions taken for this kind of branch instruction. However, if the TAGE memory predicts the branch will be taken, the CPU still cannot predict the target. To solve this problem, a module named BTB can be added, as mentioned by Burcea [9], Vandierendonck [10], and Kumar [11, 12], to store the branch target address. Unfortunately, the target of an indirect branch instruction is the sum of a general register and an offset. The offset value is defined in the instruction, so we can obtain it during the decode step, whereas the general register's value may change whenever the instruction is rerun. To enhance prediction accuracy, indirect BTB (IBTB, also called AVM-BTB [13]) is used to store the target of indirect branch instructions. IBTB is a special BTB used only to predict the jumping target of indirect branches. IBTB also includes the value of the general register taken, which enhances prediction accuracy.

Predicting branch instruction type 1 is not necessary, because the direction is always jumping without any condition. However, that of types 2 and 3 is not. However, since type 4 needs to predict direction and target, its prediction accuracy is the lowest.

Other instructions are non-branch instructions, such as calculation, load and store, and atomic instructions, which only cause the PC + 4 fetch to fetch the next instruction. These kinds of instructions do not change the PC of the next instruction, and the next PC will always be the current PC + 4. For this reason, the CPUs do not need to do branch prediction for this type of instruction. About 80% of instructions are non-branch instructions. The article will focus on branch instructions rather than non-branch instructions.

1.4. Data and instruction prefetching and runahead

Modern high-performance processors rely heavily on deep pipelines and superscalar execution to exploit instruction-level parallelism. However, their performance is often constrained by the front-end bottleneck, where instruction fetches stall due to instruction cache misses or where branch mispredictions deprive the execution engine of a steady stream of work. To mitigate this, sophisticated instruction prefetchers are employed to proactively fetch instructions from memory into the cache hierarchy before they are demanded by the program counter.

The landscape of instruction prefetching is broadly divided into two complementary philosophies: (1) Fetch-Directed Prefetching, which predicts future instruction addresses based on the control flow of the program, and (2) Execution-Guided Prefetching, exemplified by Runahead execution, which leverages the out-of-order execution engine to dynamically discover and prefetch future instructions and data during long-latency stalls.

Fetch-Directed Instruction Prefetching (FDIP) represents a seminal approach in the first category. It repurposes the branch prediction hardware (the Branch Target Buffer, or BTB) to generate a stream of future instruction block addresses, effectively decoupling instruction prefetching from the core's fetch stage. The work "Fetch-Directed Instruction Prefetching Revisited" [14] demonstrates that intelligent management of BTB metadata (tags, target addresses, and fall-through information) can significantly enhance its coverage for complex server workloads, establishing a cost-effective baseline for control-flow-based prefetching. However, FDIP fundamentally relies on the accuracy and coverage of the BTB. Recent analyses, as discussed in works such as "UDP: Utility-Driven Fetch Directed Instruction Prefetching" [15], reveal that for modern data center applications with massive, irregular code footprints, BTB misses remain a primary bottleneck. UDP advances the state of the art by introducing utility metrics to prioritize timely and accurate prefetches, thereby preventing cache pollution and bandwidth waste. It underscores a critical evolution:

moving from purely coverage-oriented prefetching to utility- and timeliness-aware prefetching.

Runahead Execution tackles the front-end bottleneck from a different, more dynamic angle. It allows the processor to enter a speculative "pre-execution" mode when a long-latency load (e.g., a last-level cache miss) stalls the main pipeline. In this mode, the processor speculatively executes subsequent instructions, discovering future memory accesses—including instruction addresses—that can be prefetched. The foundational concept of "Continuous Runahead: Transparent Hardware Acceleration" [16] showed how extending the Runahead window and filtering the instruction stream could dramatically improve prefetch coverage and accuracy for memory-bound workloads. Building upon this, "Precise Runahead Execution (PRE)" [17] introduced a major microarchitectural refinement. By reusing the renamer to track dependencies without committing architectural state, PRE achieves longer, more effective pre-execution periods with lower overhead, making the technique more practical and performant. Runahead's key strength is its ability to uncover irregular, data-dependent control and memory flows that are invisible to static predictors like the BTB.

These works collectively illuminate the current frontier and inherent limitations. FDIP-based methods are efficient and proactive but are ultimately constrained by the static predictability of the BTB. Runahead methods are powerful and adaptive to dynamic program behaviors but incur non-trivial hardware complexity and energy costs due to redundant speculative execution. Our proposed algorithm is motivated by the need to synthesize the strengths of both paradigms. Insights from UDP guide our design towards utility-driven prioritization, ensuring prefetch resources are allocated to the most performance-critical streams. The architectural efficiency principles from PRE inform our mechanism to minimize overhead. Simultaneously, we draw inspiration from the dynamic execution philosophy of Continuous Runahead to handle unpredictable control flow while avoiding its full hardware cost. By explicitly positioning our work against the contributions and limitations identified in these key studies, we aim to develop a hybrid, low-overhead prefetcher that delivers the coverage of a Runahead-inspired mechanism with the efficiency of an enhanced FDIP-style predictor, thereby offering a novel solution to the persistent challenge of front-end stalls in next-generation processors.

1.5. Data prefetching pipeline

The CPU processes the instruction and data [18] in several cycles. The pipeline architecture [19] is shown as follows:

The CPU pipeline architecture shown in **Figure 5** is a classic example of a high-performance CPU; most high-performance CPUs use a similar design. The IFU focused on the Instruction Fetch (IF) operation. The prediction information is sent out in Instruction Fetch Stage 5 (IF5) or earlier, while the data is used in Execute Stage 1 (EX1) or later. Instruction execution (IEX) is a module that focuses on CPU instruction execution. There are at least four stages between IF5 and EX1. Between IFU and IEX, there are three Decode (DC) stages and one Issue Stage (IS). For this reason, the CPU has four cycles (3 DC cycles + 1 IS cycle) to process prediction information and prefetch data.

To improve IEX efficiency by more accurately prefetching data into DCache, a better CPU microarchitecture is proposed. This architecture may enhance CPU performance by prefetching data into DCache using information from IFU, so IEX can execute instructions with fewer data misses in DCache. From **Figure 6**, when the IEX receives the prefetch data, the IFU already has branch-prediction information for four cycles. In these cycles, the CPU can refine the information and prefetch data into L1 DCache via the instruction prefetcher (IPF), and IEX can then use the prefetched data. The black line in **Figure 6** shows the common method of fetching data; the path goes through DDR, L3 DCache, L2 DCache, and L1 DCache. Three steps require several cycles to complete. However, if the IPF receives the branch prediction information mentioned earlier (the blue line shown in **Figure 6**), it can prefetch the data directly from DDR, L3 DCache, or L2 DCache. If the data is in there, the time saved is dramatic. The red line in **Figure 6** shows a possible path to prefetch the data into L1 DCache so that IEX can access it directly.

1.6. Application

High-performance CPUs are widely deployed across a broad range of application domains, including industrial and agricultural devices, automotive electronics, multimedia devices, and communication systems. In recent years, they have also become a fundamental computing platform for advanced artificial intelligence (AI) workloads, particularly for large-scale model training and inference.

AI workloads are typically characterized by a large volume of iterative computations, commonly implemented using for-loops, as well as the parallel processing of large volumes of homogeneous data. The rapidly growing demand for AI computing capability has therefore imposed new challenges on the optimization of CPU algorithms and architectural designs. Improving the efficiency of computations dominated by extensive for-loop structures has emerged as a critical research problem in both academia and industry.

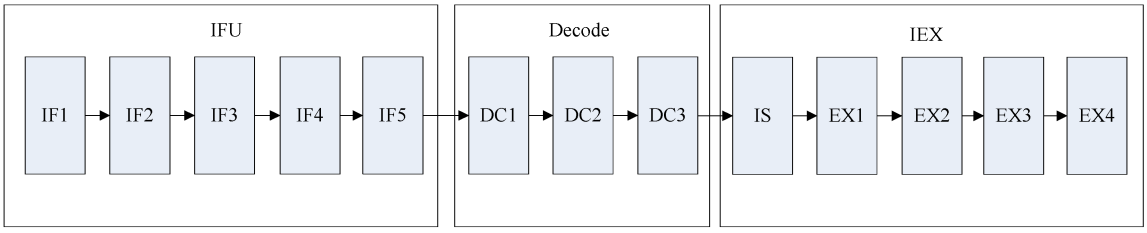


Figure 5 • Central Processing Unit (CPU) pipeline architecture. IFU: Instruction Fetch Unit; IF1–IF5: Instruction Fetch Level 1–5; DC1–DC3: Decode level 1–3; IEX: Instruction Execute; IS: Issue Stage Level; EX1–EX4: Execute Level 1–4; arrows: Data flow.

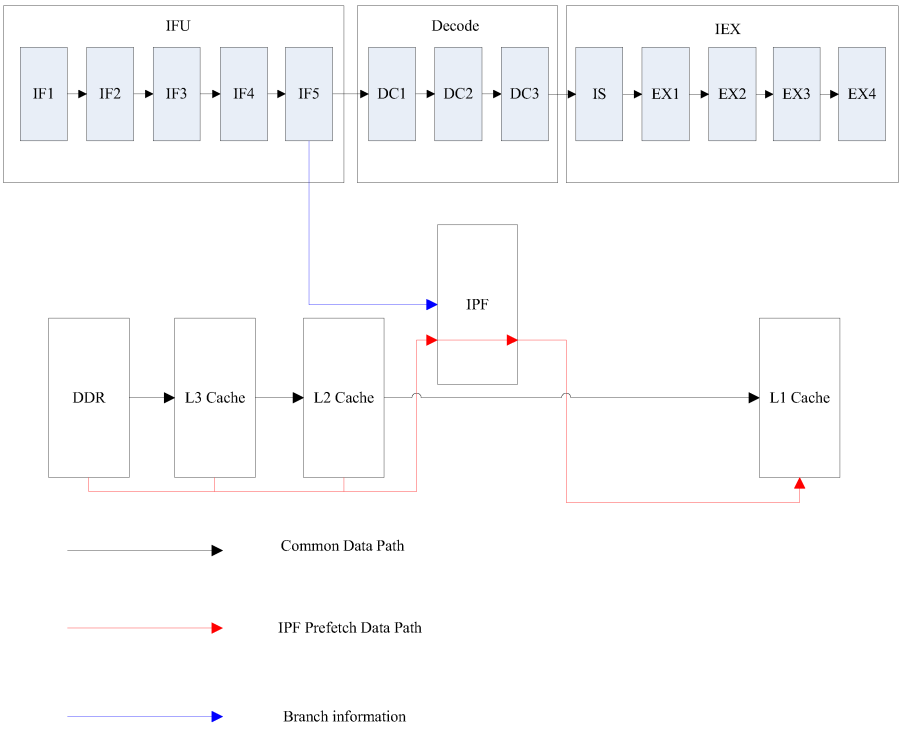


Figure 6 • Central Processing Unit (CPU) architecture with Instruction Prefetcher (IPF). IF1–IF5: Instruction Fetch Level 1–5; DC1–DC3: Decode level 1–3; IEX: Instruction Execute; IS: Issue Stage Level; EX1–EX4: Execute Level 1–4; IFU: Instruction Fetch Unit; DDR: Double Data Rate Synchronous Dynamic Random-Access Memory; L1–L3: Level 1–3.

To address the new demands of AI, researchers worldwide have conducted in-depth, innovative studies on CPU cache prefetching technology, publishing a series of high-impact results at top computer architecture conferences and in journals, and leading the latest research trends in the field. Current research has broken through the limitations of traditional prefetching technologies that rely on fixed-stride prediction and simple temporal and spatial locality, advancing in multiple cutting-edge directions. In terms of high-precision hardware prefetcher design for multi-level cache hierarchies, researchers have proposed dedicated optimizations for the L1/L2 cache to improve prediction accuracy while controlling hardware overhead, such as the Signature Pattern Prediction and Access-Map (SPPAM) prefetcher for L2 cache anti-interference optimization [20], the lightweight Local Delta Prefetcher for the L1 DCache [21], and the Dynamic Multi-Pattern Perceptron Prefetcher (DMP) for irregular memory access [22]; the cache level prediction mechanism has also realized efficient collaborative prefetching across multi-level cache hierarchies [23]. For AI-driven intelligent prefetching, the integration of reinforcement learning and neural models has become a mainstream direction, with the Athena framework realizing adaptive coordination of global prefetching through online reinforcement learning [24], and the hierarchical neural model prefetcher laying the foundation for neural-network-based memory-access address prediction [25]. Prefetcher coordination and management have become a research hotspot, where the Ignite Positive Prefetchers (I-POP) mechanism dynamically controls multiple prefetchers by quantifying prefetch effectiveness [26], and the pattern-merging strategy reduces prefetch storage overhead and improves matching efficiency through clustering algorithms [27]. In the field of server-oriented software–hardware collaborative prefetching, hierarchical prefetching based on software function bundle analysis and hardware replay has broken through the front-end bottleneck of large instruction working sets [28], and critical slice prefetching has optimized prefetching efficiency by focusing on the core execution flow of programs [29]. For Reduced Instruction Set Computer V (RISC-V) architecture-specific customized prefetching optimization, researchers have developed a dedicated prefetch performance simulation ecosystem [30] and designed a split cache with a dedicated vector data prefetch module for RISC-V vector extensions [31], thereby filling the gap in cache prefetching optimization for open-architecture processors.

AI Industrial Engineering primarily encompasses two directions of AI training and inference, both of which demand substantial computational power. While AI-specific processors are a strong option, many general-purpose computing systems, such as typical personal computers, lack them. Therefore, CPUs are also among the optimal choices for AI computation. The CPU design method proposed in this paper is applicable to both AI training/inference and general-purpose CPUs. Because this algorithm is specifically designed for data prefetching in for-loop structures, and AI training heavily relies on for-loops, the algorithm demonstrates particularly pronounced effectiveness when the CPU is engaged in AI training and inference tasks.

1.7. Research gap and aim

Existing mainstream data prefetching algorithms mainly rely on the recurrence and locality of historical data access patterns, which can deliver moderate performance gains but fail to exploit the ultimate potential of data prefetching. Building upon conventional classic prefetching schemes, this paper proposes an advanced optimized algorithm design by considering branch prediction information to reduce the research gap in this area.

Different from prior approaches, the proposed method not only considers historical data usage behavior but also incorporates branch prediction information. It establishes an association between branch instructions and load/store memory operations and fully accounts for the influence of branch prediction results on data prefetching decisions. This design effectively improves data prefetch accuracy, reduces unnecessary data movement, and lowers the power consumption. Consequently, the proposed scheme improves CPU IPC performance while achieving better power efficiency.

2. Materials and methods

In some cases, the computer program may enter a loop, repeatedly running the same part of the program. In these cases, the running program is identical, so that prefetching data can provide significant benefits.

When there is a loop in the program, data prefetching can provide significant promotion. IFU can detect a loop in the program. If an IFU detects a loop, it sends this information to IPF, and IPF can then prefetch data from the high-level cache into the L1 cache. For this case, a new algorithm is proposed.

The algorithm of loop program identification is as follows:

Define a table named BHT (branch history table) to record all direct branch instructions and their PC.

Each page has a counter to record the number of repetitions of this branch instruction.

When a branch instruction is executed, and the operation is taken, its counter is incremented, and another branch counter is cleared to 0.

When the counter value reaches a threshold, the program enters a loop, and IFU can detect IPF for the next operation.

A usage counter is defined. When the BHT is full, the branch with the lowest usage counter is found and replaced with the next available branch.

The details of the algorithm are described as follows (**Figure 7**):

(**a**)

	Branch PC	Repeat counter	Usage counter
entry0	Branch 0 PC	0	0
entry1	Branch 1 PC	0	0
entry2	Branch 2 PC	0	0
entry3	Branch 3 PC	0	0
entry4	Branch 4 PC	0	0
entry5	Branch 5 PC	0	0
entry6	Branch 6 PC	0	0
entry7	Branch 7 PC	0	0

(**b**)

	Branch PC	Repeat counter	Usage counter
entry0	Branch 0 PC	1	1
entry1			
entry2			
entry3			
entry4			
entry5			
entry6			
entry7			

(**c**)

	Branch PC	Repeat counter	Usage counter
entry0	Branch 0 PC	1	1
entry1	Branch 1 PC	1	1
entry2	Branch 2 PC	1	1
entry3	Branch 3 PC	1	1
entry4			
entry5			
entry6			
entry7			

(**d**)

	Branch PC	Repeat counter	Usage counter
entry0	Branch 0 PC	1->0	1
entry1	Branch 1 PC	1->2	1->2
entry2	Branch 2 PC	1->0	1
entry3	Branch 3 PC	1->0	1
entry4			
entry5			
entry6			
entry7			

(**e**)

	Branch PC	Repeat counter	Usage counter
entry0	Branch 0 PC	0	1
entry1	Branch 1 PC	2	2
entry2	Branch 2 PC	0->1	1->2
entry3	Branch 3 PC	0->1	1->2
entry4			
entry5			
entry6			
entry7			

(**f**)

	Branch PC	Repeat counter	Usage counter
entry0	Branch 0 PC	0	1
entry1	Branch 1 PC	2->3	2->3
entry2	Branch 2 PC	1->0	2
entry3	Branch 3 PC	1->0	2
entry4			
entry5			
entry6			

Figure 7 • *Cont.*

	Branch PC	Repeat counter	Usage counter
entry0	Branch 0 PC	0	1
entry1	Branch 1 PC	3	3
entry2	Branch 2 PC	0->1	2->3
entry3	Branch 3 PC	0	2
entry4			
entry5			
entry6			
entry7			

	Branch PC	Repeat counter	Usage counter
entry0	Branch 0 PC	0	1
entry1	Branch 1 PC	3	3
entry2	Branch 2 PC	1	3
entry3	Branch 3 PC	0->1	2->3
entry4			
entry5			
entry6			
entry7			

(**g**)

	Branch PC	Repeat counter	Usage counter
entry0	Branch 0 PC	0	1
entry1	Branch 1 PC	3->4	3->4
entry2	Branch 2 PC	1->0	3
entry3	Branch 3 PC	1->0	3
entry4			
entry5			
entry6			
entry7			

(**h**)

	Branch PC	Repeat counter	Usage counter
entry0	Branch 0 PC	0	1
entry1	Branch 1 PC	0	1
entry2	Branch 2 PC	4	4
entry3	Branch 3 PC	0	4
entry4	Branch 4 PC	0	4
entry5	Branch 5 PC	0	4
entry6	Branch 6 PC	0	4
entry7	Branch 7 PC	0	4

(**i**)

	Branch PC	Repeat counter	Usage counter
entry0	Branch 0 PC	0	1
entry1	Branch 1 PC	0	1
entry2	Branch 2 PC	4	4
entry3	Branch 3 PC	0	4
entry4	Branch 4 PC	0	4
entry5	Branch 5 PC	0	4
entry6	Branch 6 PC	0	4
entry7	Branch 7 PC	0	4

(**j**)

	Branch PC	Repeat counter	Usage counter
entry0	Branch 0->8 PC	1	1
entry1	Branch 1 PC	0	1
entry2	Branch 2 PC	4	4
entry3	Branch 3 PC	0	4
entry4	Branch 4 PC	0	4
entry5	Branch 5 PC	0	4
entry6	Branch 6 PC	0	4
entry7	Branch 7 PC	0	4

(**k**)

	Branch PC	Repeat counter	Usage counter
entry0	Branch 8 PC	1	1
entry1	Branch 1 PC	0	1->0
entry2	Branch 2 PC	4	4->3
entry3	Branch 3 PC	0	4->3
entry4	Branch 4 PC	0	4->3
entry5	Branch 5 PC	0	4->3
entry6	Branch 6 PC	0	4->3
entry7	Branch 7 PC	0	4->3

(**l**)

	Branch PC	Repeat counter	Usage counter
entry0	Branch 8 PC	1	1->0
entry1	Branch 1->9 PC	1	1
entry2	Branch 2 PC	4	3->2
entry3	Branch 3 PC	0	3->2
entry4	Branch 4 PC	0	3->2
entry5	Branch 5 PC	0	3->2
entry6	Branch 6 PC	0	3->2
entry7	Branch 7 PC	0	3->2

(**m**)

	Branch PC	Repeat counter	Usage counter
entry0	Branch 8 PC	1	0
entry1	Branch 9 PC	1	1
entry2	Branch 2 PC	4->5	2->3
entry3	Branch 3 PC	0	2
entry4	Branch 4 PC	0	2
entry5	Branch 5 PC	0	2
entry6	Branch 6 PC	0	2
entry7	Branch 7 PC	0	2

(**n**)

Figure 7 • The details of the algorithm. (**a**) The architecture of branch history table (BHT); (**b**) Branch 0 found; (**c**) Branches 1–3 found; (**d**) Branch 1 found twice; (**e**) Branches 2–3 found again; (**f**) Branch 1 found for 3rd time; (**g**) Branches 2–3 found 3rd time; (**h**) Branch 1 found for 4th time—The repeat counter match the threshold 4 can be identified as a loop; (**i**) branch history table (BHT) has 8 entries (full); (**j**) Replace entry 0 (Branch 0) with the least usage counter—When a new branch is found but the BHT is full, the branch with less usage counter will be replaced; (**k**) Branch 8 initialized in entry 0—Branch 0 -> Branch 8; (**l**) Usage counter of other entries minus 1; (**m**) Branch 9 initialized in entry 1—When another branch is found but the BHT is full, the branch with less usage counter will be replaced—Branch 1 -> Branch 9, the usage counter of other branch minus 1 at the same time; (**n**) Branch 2 hit again—When an old branch is hit, no branch will be replaced, the usage counter and repeat counter of this entry add 1. Red color: the data in BHT is changed; Blue color: the data in BHT is not changed.

Define a BHT block in the IFU, and assume its size is 8, meaning it can store up to 8 branches. In real CPU design, it will be much larger.

When a branch is found, the branch PC is added to the BHT, and its repeat counter and usage counter are set to 1. The word in red means the data in BHT is changed while the word in black means the data in BHT is not changed.

For the other taken branches (branches 1–3), 1 entry is allocated to each.

When branch 1 is found again, its repeat counter is set to 2, while the repeat counters of the other branches are set to 0 (1 − 1 = 0). This operation means the branch repeats twice; other branches have not been found yet, so 1 is subtracted from their repeat counters.

When branches 2 and 3 are found again, their repeat and usage counters are increased by 1.

However, when branch 1 is found the third time, the repeat counter of branch 1 will add 1, but the repeat counters of branch 2 and branch 3 will subtract 1 and become 0 again.

When branch 2 and branch 3 are found for the third time, their repeat counters increase by 1, and their usage counters change to 3 (2 + 1 = 3).

In this case, branches 1–3 are all found 3 times, but the algorithm only wants to focus on the first branch, so the repeat counter of branch 1 is 3, but the repeat counters of branch 2 and branch 3 are still 0. This algorithm is designed to find the 1st taken branch (Branch 1) in a loop, while other taken branches are gradually ignored.

When branch 1 is found for the 4th time, its repeat counter reaches 4, which meets the threshold. At this time, the hardware can detect a loop in branch 1.

However, the size of BHT is limited; an algorithm is necessary to replace the useless branch when the BHT is full. **Figure 7i** shows a status where BHT with 8 entries is full.

The first entry is replaced with the one with the lowest usage counter, as shown in **Figure 7j**. Entry 0 with branch 0 is replaced. This entry is in blue which means it was chosen to be replaced.

When the value at entry 0 is replaced with branch 8, the repeat counter and usage counter are initialized to 1.

Since 1 branch is deleted and initialized from another branch, the usage of the other counter is −1. This means that the use of other entries decreases.

When another branch (branch 9) is found, the BHT is full. Another entry must be found to replace this. Entry 1 with branch 1 is replaced, as it has the lowest usage counter. At the same time, the usage counter of other branches will be minus 1 for the same reason.

When an old branch is hit, the operation is the same as before, and no entry will be initialized. The usage counter for branch 2 will increase by 1 (2 + 1 = 3).

With this algorithm, the new branch still has a chance to enter the BHT, but the most significant branch will always remain in the BHT. Then the program's loop will be easier to find, leading to better CPU performance.

After a loop is found in a program, data can be prefetched in the following steps:

Step 1: Record the data address loaded with the first instruction in one loop cycle. This address is defined as the First Instruction Data Address (FIDA).

Step 2: Find the step length of FIDA between each loop cycle and record its length. This length is defined as the Data Address Loop Distance (DALD).

Step 3: If the DALD is the same across several loop cycles (i.e., the number is the threshold for loop definition), it is assumed that this loop program uses data with special rules. For this reason, data is prefetched with these rules in the next loop cycles.

Step 4: Data is prefetched before the next loop cycle with the DALD, and then prefetched into the DCache before the LSU uses it, thereby enhancing CPU performance by reducing the DCache miss rate.

3. Results

3.1. Algorithm equation

The new algorithm is recommended with some equations, as follows (summarize from **Figure 7**):

Definition:

Allocate: When the program executes a branch instruction, and the branch is written in as an entry in the BHT (branch history table) for the first time, it is called "allocate," as shown in **Figure 7b**.

Hit: When a branch is in the BHT, the program re-executes the branch instruction; this is called a "hit", as shown as branch1 in **Figure 7d**.

When the BHT is full, one entry is selected to delete the old branch information and write the new branch information into that entry. For the new branch, it can also be called "allocate". For the BHT, it is called "replace". This is shown as Branch 0 is replaced by Branch 8 in **Figure 7k**.

The method to update the repeat counter (REP) is as follows:

If allocated: REP = 0

If hit: REP = REP + 1

If hit other entry and its REP > 1, REP = REP − 1

Else: REP = REP

The method to update the usage counter(U):

If allocated: REP = 0

If hit: REP = REP + 1

If other entries are replaced, REP = REP − 1

Else: REP = REP

Replace Entry Select (RES):

RES = GEIDX(Min(Uo, U1, ..., Un))

If with the same U, get the entry with the least REP number: RES = GEIDX(Min(REPo, REP 1, ..., REP n)

If with the same U and RES, get the entry with the least Entry Index: RES = MIN(EIDXo, EIDX 1, ..., EIDX n)

GEIDX: Get entry index of the selected value

EIDX: Entry index

Loop definition:

When the Repetition number exceeds a threshold, it is considered a loop: REP > TH.

Loop Operation:

When a loop is found in a program, data can be prefetched into the DCache, as mentioned in Section 2.

When the repeat counter value exceeds a threshold, it is concluded that a loop function has been found in the program. Then, data is prefetched into DCache to accelerate the loop program by increasing the DCache hit rate. With this new algorithm, the CPU's performance with the new algorithm can be slightly improved, as mentioned in Section 3.2.

3.2. Simulation results

This paper presents three simulation results obtained from the Gem5 simulation platform: (1) a simple prefetching algorithm; (2) a stride-based prefetching algorithm; and (3) an optimized stride prefetching algorithm based on branch prediction information. The simulation results for the three algorithms are shown in **Table 2**, the key performance indicator is Instructions Per Cycle (IPC):

Table 2 • Simulation results between the optimized algorithm and the traditional algorithm.

Case	Algorithm	IPC	Cache miss times	Cache access times	Cache miss rate	Delta IPC	Delta Cache miss rate
1	traditional algorithm	1.231	102,824	1,516,179	0.06781785	N/A	N/A
	optimized algorithm	1.448	92,194	1,519,472	0.060675024	0.217	−0.0071428260
2	traditional algorithm	0.408	133,851	3,170,901	0.042212292	N/A	N/A
	optimized algorithm	1.451	2799	3,174,147	0.000881812	1.043	−0.0413304806
3	traditional algorithm	0.257	182,991	385,552	0.474620803	N/A	N/A
	optimized algorithm	0.27	179,045	388,782	0.460528008	0.013	−0.0140927954
4	traditional algorithm	0.445	133,887	3,170,924	0.042223339	N/A	N/A
	optimized algorithm	1.513	4763	3,174,105	0.00150058	1.068	−0.0407227588
5	traditional algorithm	0.586	130,057	4,222,749	0.030799131	N/A	N/A
	optimized algorithm	1.48	905	4,219,572	0.000214477	0.894	−0.0305846544
6	traditional algorithm	0.552	252,761	8,413,978	0.030040606	N/A	N/A
	optimized algorithm	1.294	779	8,417,143	9.25492×10^{-5}	0.742	−0.0299480570
7	traditional algorithm	0.728	2851	58,093	0.049076481	N/A	N/A
	optimized algorithm	1.005	387	61,316	0.006311566	0.277	−0.0427649145
8	traditional algorithm	1.349	2805	230,146	0.012187915	N/A	N/A
	optimized algorithm	1.523	462	233,380	0.001979604	0.174	−0.0102083114
9	traditional algorithm	1.176	2975	162,148	0.018347436	N/A	N/A
	optimized algorithm	1.399	480	165,379	0.002902424	0.223	−0.0154450115
10	traditional algorithm	1.451	134,127	4,258,951	0.031492966	N/A	N/A
	optimized algorithm	1.516	82,774	4,259,965	0.019430676	0.065	−0.0120622902

IPC: Instructions Per Cycle.

The traditional algorithm is the common DCache prefetching algorithm given by the Gem5 default setting, while the optimized algorithm is introduced in Section 3.1. The delta IPC and the delta cache-miss rate are calculated as the difference between the optimized and traditional algorithms. The data in **Table 2** show that the IPC and cache-miss rate of the optimized algorithm are better than those of the traditional algorithm.

The mean of delta IPC is as follows:

$$mean = \frac{\sum_{i=1}^{10} x_i}{10} = 0.4716 \quad (1)$$

The standard variance in delta IPC is as follows:

$$S^2 = \frac{\sum_{i=1}^{10} (x_i - mean)^2}{10} = 0.15624244 \quad (2)$$

The standard deviation of delta IPC is as follows:

$$S = \sqrt{\frac{\sum_{i=1}^{10} (x_i - mean)^2}{10}} = 0.395275145 \quad (3)$$

For 95% confidence, the confidence margin is as follows:

$$Confidence\ margin = 0.244989571 \quad (4)$$

The confidence interval of the delta IPC is as follows:

$$Confidence\ interval = mean \pm Z \times S = 0.4716 \pm 0.244989571 = [0.226610429 - 0.716589571] \quad (5)$$

By the same algorithm, the article can introduce the confidence interval of the delta cache missing rate. The statistical data of IPC and confidence interval are shown in **Table 3**:

Table 3 • Statistical data between the optimized algorithm and the traditional algorithm.

Item	Instructions Per Cycle (IPC)	Cache miss rate
Mean	0.4716	−0.02443021
Standard deviation	0.395275145	0.013406077
Variance	0.15624244	0.00017972
Sample number	10	10
Confidence margin	0.244989571	0.00830902
Minimum	0.226610429	−0.0327392297
Maximum	0.716589571	−0.0161211903

Therefore, we can reasonably conclude from the statistical data that the observed results are not due to simulation noise or statistical errors.

Another very common algorithm for DCache prefetching is the stride algorithm. The stride algorithm is one of the core algorithms of data stream prefetchers. This article also compares the optimized and traditional algorithms, as shown in **Table 4**.

Using the same algorithms and equations mentioned in the previous part, this article presents the mean, standard deviation, variance, sample size, confidence margin, and confidence interval for the optimized algorithm and the stride algorithm, as shown in **Table 5**.

Table 4 • Simulation results between the optimized algorithm and the stride algorithm.

Case	Algorithm	IPC	Cache miss times	Cache access times	Cache miss rate	Prefetch time	Prefetch useful time	Prefetch useful rate	Delta IPC	Delta Cache miss rate	Prefetch useful rate
1	stride-based	0.527202	2461	73,266	0.033589933	1584	213	0.134469697	N/A	N/A	N/A
	optimized stride	0.518865	2508	73,276	0.034226759	413	165	0.399515574	−0.00834	0.00063683	0.26504588
2	stride-based	2.09425	97,364	1,532,108	0.063549045	187,619	5675	0.030247	N/A	N/A	N/A
	optimized stride	2.098521	99,602	1,532,119	0.065009311	75,462	3420	0.0453208	0.004271	0.00146027	0.0150738
3	stride-based	1.270107	2456	245,297	0.010012352	1423	167	0.117357695	N/A	N/A	N/A
	optimized stride	1.265786	2489	245,307	0.01014647	383	136	0.355091384	−0.00432	0.00013412	0.23773369
4	stride-based	0.40053	2496	47,326	0.052740565	1572	167	0.426020408	N/A	N/A	N/A
	optimized stride	0.396181	2526	47,336	0.053363191	392	136	0.346938776	−0.00435	0.00062263	−0.07908163
5	stride-based	0.371795	2459	44,599	0.055135765	1423	167	0.117357695	N/A	N/A	N/A
	optimized stride	0.367815	2489	44,609	0.055795916	383	136	0.355091384	−0.00398	0.00066015	0.23773369
6	stride-based	0.692301	2603	87,762	0.029659762	2721	454	0.166850422	N/A	N/A	N/A
	optimized stride	0.671907	2597	87,772	0.029588024	712	459	0.644662921	−0.02039	−0.00007174	0.4778125

IPC: Instructions Per Cycle.

Table 5 • Statistical data between the optimized algorithm and the stride algorithm.

Item	Instructions Per Cycle (IPC)	Cache miss rate	Prefetch useful rate
Mean	−0.006185	0.000573708	0.1938632
Standard deviation	0.00738875	0.000484712	0.180736854
Variance	0.00005459	0.00000023	0.03266581
Sample number	6	6	6
Confidence margin	0.00591213	0.000387843	0.144616945
Minimum	−0.0120971	0.000185865	0.047769375
Maximum	−0.0002729	0.000961551	0.337003265

By analyzing the simulation results and statistical data, this article concludes that the stride algorithm may offer very little benefit in IPC and cache miss rate, but the optimized algorithm offers significant benefit in the prefetch useful rate. In other words, compared to the stride algorithm, the optimized algorithm introduced in this article sacrifices very little IPC but gains a significant benefit by prefetching very little data. The prefetching hit rate is enhanced by branch prediction information, because the optimized algorithm uses it to predict useful data and prefetch it into DCache. With this new algorithm, the CPU can prefetch less data to avoid cache pollution and save significant power while only losing very little IPC. The new algorithm still has greater benefits than the stride algorithm in most CPU application scenarios.

According to the simulation result and statistical data table, the latest optimization algorithm achieves the goal of improving IPC compared to the other algorithms by enhancing data prefetching accuracy and reducing the cache miss rate. The increase in data prefetching accuracy reduces unnecessary data movement compared to stride algorithms, which also helps save power. Power consumption is a key performance indicator for the efficiency of AI training and inference.

4. Discussion

The new algorithm and architecture can improve CPU speed by more accurately prefetching data into DCache. The key idea of this research is not only to exploit time and spatial dependencies but also to use branch-predictor results to prefetch DCache. With more information, the accuracy of DCache data prefetching can be enhanced. Another innovation point is the special replacement policy algorithm in BHT, which can better recognize loop functions. When the algorithm detects a loop, it can more effectively prefetch DCache data, especially for programs with many loop iterations.

As deeper research emerges, more methods should be explored to reduce power consumption in the future by using improved prefetching techniques. If the power can be reduced using the same methods, the new algorithm and architecture can be widely adopted in portable electronic devices' CPUs.

The new algorithm and architecture optimization enhance performance for both general-purpose computing and AI-specific algorithms. The rationale is that AI training and reasoning algorithms often involve extensive parallel for-loops, using instructions such as ARM-Neon (Advanced RISC Machine Neon) or RISC-V RVV (RISC-V vector extension) to achieve large-scale data parallelism. These computations tend to rapidly consume data in DCache, causing DCache to become a significant performance bottleneck. Our approach specifically optimizes for-loop operations, thereby reducing the DCache miss rate and naturally improving performance during AI training and inference. This new algorithm and architecture are designed to enhance CPU performance, especially for AI operations involving large loops.

For further applications, this advanced CPU can be integrated into smart electronic devices for smart industry and agricultural applications, leveraging new AI algorithms. This smart electronic device with this CPU can include an AI function and improve performance while using less power.

In future work, we may simulate additional CPU performance parameters. In addition to focusing on improvements in Instructions Per Cycle (IPC), other metrics, such as the cache miss rate and DCache data prefetch efficiency (prefetch hit rate), can also be examined. Improvements in these parameters would provide stronger empirical support for the arguments presented in this paper. Accordingly, subsequent research will attempt to incorporate the simulation of these additional performance metrics within the Gem5 CPU simulation platform.

5. Conclusions

The instruction prefetcher (IPF) design can accelerate data fetches from IEX using branch-prediction data. The refining and prefetching algorithm can be optimized further to improve performance. The IPF only contains combinational and sequential logic to run the refining and prefetching algorithms; these components occupy a very limited area. Power consumption will be discussed in a future paper.

This section quantitatively summarizes the overall performance advantages of the proposed prefetching scheme over stride-based data prefetchers. Experimental results demonstrate that the optimized algorithm effectively reduces redundant data prefetch operations, yielding a noticeable improvement in prefetch accuracy compared with conventional stride prefetching methods. Assisted by branch prediction information, the proposed approach reduces the DCache miss rate, especially for loop-heavy program workloads. Meanwhile, the refined BHT replacement policy curtails unnecessary data migration, lowering memory bandwidth usage and power consumption while maintaining nearly unchanged IPC performance with negligible performance loss.

Unlike traditional mainstream prefetching techniques that only exploit temporal and spatial locality of historical data access, this work innovatively couples branch instruction prediction with load/store memory operations. By fully incorporating the impact of branch prediction outcomes on data prefetching decisions and introducing loop detection based on dedicated counter mechanisms, the method can effectively identify complex loop structures that are difficult to capture by traditional prefetchers. Relative to the current state of the art, the proposed algorithm achieves an improved balance among prefetch accuracy, redundant data movement suppression, and power efficiency. It provides a lightweight, low-overhead microarchitectural optimization suitable for modern general-purpose processors as well as AI training and inference workloads dominated by parallel loop computation.

In summary, for the comparison between the traditional algorithm and optimized algorithm, the optimized algorithm can improve the IPC by 0.4716 instructions per cycle and reduce the cache miss rate by 2.443% (results from **Table 3**). The optimized algorithm has better performance than the traditional algorithm.

Acknowledgments

The authors wish to thank Yao Yongbin for writing the book "Super Scalar Processor Design", which provided valuable insights into CPU architecture. The authors also wish to thank Wang Jian for their technical assistance with the Gem5 simulation platform and simulation results generation.

Funding

This research received no external funding.

Author contributions

Conceptualization, D.N. and R.W.M.K.; methodology, D.N. and R.W.M.K.; software, D.N.; validation, D.N., R.W.M.K. and S.P.F.; formal analysis, D.N.; investigation, D.N.; resources, D.N.; data curation, D.N.; writing—original draft preparation, D.N.; writing—review and editing, R.W.M.K.; visualization, D.N.; supervision, S.P.F. All authors have read and agreed to the published version of the manuscript.

Conflict of interest

Ray Wai Man Kong is a modernization director at Eagle Nice (International) Holdings Ltd. (Hong Kong, China). The employer played no role in the study's conception, design, execution, or interpretation, or the writing of this manuscript. The authors declare no other financial or personal competing interests.

Data availability statement

The data supporting the findings of this publication can be made available upon request.

Additional information

Received: 21 December 2025

Accepted: 21 May 2026

Published: 10 June 2026

Academia Engineering papers should be cited as *Academia Engineering 2026*, ISSN 2994-7065, https://doi.org/10.20935/AcadEng8332. The journal's official abbreviation is *Acad. Eng*.

Publisher's note

Academia.edu Journals stays neutral with regard to jurisdictional claims in published maps and institutional affiliations. All claims expressed in this article are solely those of the authors and do not necessarily represent those of their affiliated organizations, or those of the publisher, the editors and the reviewers. Any product that may be evaluated in this article, or claim that may be made by its manufacturer, is not guaranteed or endorsed by the publisher.

Copyright

© 2026 copyright by the authors. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

References

1. Ishii Y, Lee J, Nathella K, Sunwoo D. Re-establishing fetch-directed instruction prefetching: an industry perspective. In: Proceedings of the 2021 IEEE international symposium on performance analysis of systems and software (ISPASS); 28–30 March 2021; Stony Brook, NY, USA. Piscataway, NJ, USA: IEEE; 2021. p. 172–82. doi: 10.1109/Ispass51385.2021.00034

2. Juan T, Sanjeevan S, Navarro JJ. Dynamic history-length fitting: a third level of adaptivity for branch prediction. SIGARCH Comput Archit News. 1998; 26 (3): 155–66. doi: 10.1145/279361.279379

3. Yeh T-Y, Patt YN. Two-level adaptive training branch prediction. In: Proceedings of the 24th annual international symposium on microarchitecture (MICRO-24); 18–21 November 1991; Albuquerque, NM, USA. New York, NY, USA: Association for Computing Machinery; 1991. p. 51–61. doi: 10.1145/123465.123475

4. Ball T, Larus JR. Branch prediction for free. In: Proceedings of the ACM SIGPLAN 1993 conference on programming language design and implementation (PLDI '93); 23–25 June 1993; Albuquerque, NM, USA. New York, NY, USA: Association for Computing Machinery; 1993. p. 300–13. doi: 10.1145/155090.155119

5. Smith JE. A study of branch prediction strategies. In: Proceedings of the 25 years of the international symposia on computer architecture (selected papers); 27 June–2 July 1998; Barcelona, Spain. New York, NY, USA: Association for Computing Machinery; 1998. p. 202–15. doi: 10.1145/285930.285980

6. Seznec A. Analysis of the O-GEometric history length branch predictor. In: Proceedings of the 32nd international symposium on computer architecture (ISCA'05); 4–8 June 2005; Madison, WI, USA. Piscataway, NJ, USA: IEEE Computer Society; 2005. p. 394–405. doi: 10.1109/Isca.2005.13

7. Seznec A, Michaud P. A case for (partially) tagged geometric history length branch prediction. J Instr Level Parallelism. 2006; 8: 1–23.

8. Cai L, Deshmukh A, Lai E, Mansoorshahi A, Tatulescu M, Nudelman I, et al. TAGE-SC-L with a code structure correlator. 2025. Available from: https://ericrotenberg.wordpress.ncsu.edu/files/2025/06/cbp2025-final27-Cai.pdf

9. Burcea I, Moshovos A. Phantom-BTB: a virtualized branch target buffer design. ACM Sigplan Not. 2009; 44 (3): 313–23. doi: 10.1145/1508284.1508281

10. Vandierendonck H, Desmet V, De Bosschere K. Behavior-based branch prediction by dynamically clustering branch instructions. J Inf Sci Eng. 2008; 24 (3): 919–31.

11. Kumar R, Huang CC, Grot B, Nagarajan V. Boomerang: a metadata-free architecture for control flow delivery. In: Proceedings of the 2017 IEEE international symposium on high performance computer architecture (HPCA); 4–8 February 2017; Austin, TX, USA. Piscataway, NJ, USA: IEEE; 2017. p. 493–504. doi: 10.1109/Hpca.2017.53

12. Kumar R, Grot B. Shooting down the server front-end bottleneck. ACM Trans Comput Syst. 2021; 38 (3–4): 1–30. doi: 10.1145/3484492

13. Liu Y, Li X, Zhang T, Liu T, Guo Q, Zhang F, et al. AVM-BTB: adaptive and virtualized multi-level branch target buffer. In: Proceedings of the 2024 ACM/IEEE 51st annual international symposium on computer architecture (ISCA); 29 June–3 July 2024; Buenos Aires, Argentina. Piscataway, NJ, USA: IEEE; 2024. p. 17–31. doi: 10.1109/ISCA59077.2024.00012

14. Asheim T, Kumar R, Grot B. Fetch-directed instruction prefetching revisited. arXiv. 2020. doi: 10.48550/arXiv.2006.13547

15. Oh S, Xu M, Khan TA, Kasikci B, Litz H. UDP: utility-driven fetch directed instruction prefetching. In: Proceedings of the 51st annual international symposium on computer architecture (ISCA 2024); 29 June–3 July 2024; Buenos Aires, Argentina. Piscataway, NJ, USA: IEEE Press; 2025. p. 1188–201. doi: 10.1109/ISCA59077.2024.00089

16. Hashemi M, Mutlu O, Patt YN. Continuous runahead: transparent hardware acceleration for memory intensive workloads. In: Proceedings of the 49th annual IEEE/ACM international symposium on microarchitecture (MICRO 2016); 15–19 October 2016; Taipei, Taiwan. Piscataway, NJ, USA: IEEE; 2016. p. 1–12. doi: 10.1109/MICRO.2016.7783764

17. Naithani A, Feliu J, Adileh A, Eeckhout L. Precise runahead execution. IEEE Comput Archit Lett. 2019; 18 (1): 71–4. doi: 10.1109/LCA.2019.2910518

18. Hennessy JL, Patterson DA. Computer architecture: a quantitative approach. 6th ed. Cambridge, MA, USA: Morgan Kaufmann; 2017.

19. Miyazaki H, Kanamori T, Islam MA, Kise K. RVCoreP: an optimized RISC-V soft processor of five-stage pipelining. IEICE Trans Inf Syst. 2020; E103.D (12): 2494–503. doi: 10.1587/transinf.2020PAP0015

20. Merrell M, Wang L, Kalafatis S, Gratz PV. SPPAM: signature pattern prediction and access-map prefetcher. arXiv. 2026. doi: 10.48550/arXiv.2602.04100

21. Navarro-Torres A, Panda B, Alastruey-Benedé J, Ibáñez P, Viñals-Yúfera V, Ros A. A complexity-effective local Delta prefetcher. IEEE Trans Comput. 2025; 74 (5): 1482–94. doi: 10.1109/TC.2025.3533086

22. Wang W, Li H, Zhang P. DMDP: a dynamic multi-Delta prefetcher with customizable runtime quality-aware feedback. Res Sq. 2024. doi: 10.21203/rs.3.rs-3853411/v1

23. Jalili M, Erez M. Reducing load latency with cache level prediction. In: Proceedings of the 2022 IEEE international symposium on high-performance computer architecture (HPCA); 2–6 April 2022; Seoul, Republic of Korea. Piscataway, NJ, USA: IEEE; 2022. doi: 10.1109/HPCA53966.2022.00054

24. Bera R, Lang Z, Hengartner C, Kanellopoulos K, Kumar R, Sadrosadati M, et al. Athena: synergizing data prefetching and off-chip prediction via online reinforcement learning. arXiv. 2026. doi: 10.48550/arXiv.2601.17615

25. Shi Z, Jain A, Swersky K, Hashemi M, Ranganathan P, Lin C. A hierarchical neural model of data prefetching. In: Proceedings of the 26th ACM international conference on architectural support for programming languages and operating systems (ASPLOS 2021); 19–23 April 2021; New York, NY, USA. New York, NY, USA: Association for Computing Machinery; 2021. p. 861–73. doi: 10.1145/3445814.3446752

26. Lin Y, Lin W, Chen Y, Xu J, Cai S, Ye J, et al. I-POP: ignite positive prefetchers. In: Proceedings of the 2026 IEEE international symposium on high performance computer architecture (HPCA); 31 January–4 February 2026; Sydney, Australia. Piscataway, NJ, USA: IEEE; 2026. p. 1–16. doi: 10.1109/HPCA68181.2026.11408454

27. Jiang S, Yang Q, Ci Y. Merging similar patterns for hardware prefetching. In: Proceedings of the 2022 55th IEEE/ACM international symposium on microarchitecture (MICRO); 1–5 October 2022; Chicago, IL, USA. Piscataway, NJ, USA: IEEE; 2022. p. 1012–26. doi: 10.1109/MICRO56248.2022.00071

28. Zhang T, Grot B, He W, Lv Y, Qu P, Su F, et al. Hierarchical prefetching: a software-hardware instruction prefetcher for server applications. In: Proceedings of the 30th ACM international conference on architectural support for programming languages and operating systems (ASPLOS 2025);

30 March–3 April 2025; New York, NY, USA. New York, NY, USA: Association for Computing Machinery; 2025. p. 529–44. doi: 10.1145/3676641.3716260

29. Litz H, Ayers G, Ranganathan P. CRISP: critical slice prefetching. In: Proceedings of the 27th ACM international conference on architectural support for programming languages and operating systems (ASPLOS 2022); 28 February–4 March 2022; Lausanne, Switzerland. New York, NY, USA: Association for Computing Machinery; 2022. p. 300–13. doi: 10.1145/3503222.3507745

30. Crespo L, Neves N, Tomas P, Roma N. Advancing the RISC-V performance simulation ecosystem with data prefetching. In: Neuwirth S, Paul AK, Weinzierl T, Carson EC, editors. Proceedings of the ISC high performance 2025 conference (high performance computing); 10–13 June 2025; Hamburg, Germany. Cham, Switzerland: Springer; 2025. p. 548–61. doi: 10.1007/978-3-032-07612-0_42

31. Rebolledo S, Perez B, Bosque J, Hsu P. The bicameral Cache: a split cache for vector architectures. In: Proceedings of the 2024 IEEE 30th international conference on parallel and distributed systems (ICPADS); 3–5 December 2024; Belgrade, Serbia. Piscataway, NJ, USA: IEEE; 2024. p. 728–36. doi: 10.1109/ICPADS63350.2024.00099